

Information Security
Analyst(ALX4_ISS6_S1)odd

Project Documentation

End-to-End Vulnerability Management System

(Team 3)

1. Project Planning & Management

1.1 Project Proposal

Overview:

This project aims to build a Vulnerability Management System that identifies, analyzes, prioritizes, and remediates security vulnerabilities in IT assets.

Objectives:

- Detect vulnerabilities in systems and applications
- Evaluate risk severity using scoring systems (e.g., CVSS)
- Prioritize vulnerabilities for remediation
- Improve overall security posture

Scope:

- Includes servers, web applications, and network devices
 - Focuses on vulnerability detection and management
 - Does not include exploitation or offensive attacks
-

1.2 Project Plan

Timeline (Gantt Overview)

Phase	Duration
Planning	Week 1
Scanning	Week 2
Remediation	Week 3
Reporting	Week 4

Milestones

- Asset Discovery Completed
- Vulnerability Scan Completed
- Remediation Applied
- Final Report Delivered

Deliverables

- Asset Inventory
- Vulnerability Reports
- Remediation Plan
- Final Documentation

Resource Allocation

- Tools: Nessus / OpenVAS
- Platform: GitHub
- Lab Environment: Virtual Machines

1.3 Task Assignment & Roles

Role	Responsibility
Team Leader	Manage tasks and deadlines
Security Analyst	Analyze vulnerabilities
Scanner Specialist	Run and configure scans
Developer	Implement fixes
Documentation	Prepare reports

1.4 Risk Assessment & Mitigation

Risk	Mitigation
Tool failure	Use alternative tools

Risk	Mitigation
Lack of vulnerabilities	Use vulnerable lab (e.g., DVWA)
Time constraints	Early task distribution

1.5 KPIs

- Number of vulnerabilities detected
- Time to remediation
- Risk reduction rate
- Scan coverage

2. Literature Review

2.1 Feedback & Evaluation

- Continuous feedback from instructor during project phases
- Evaluation based on progress, implementation, and documentation

2.2 Suggested Improvements

- Improve scan accuracy
- Enhance reporting visualization
- Automate remediation processes

2.3 Final Grading Criteria

- Documentation Quality
- System Implementation
- Testing & Validation
- Final Presentation

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs
Security Team	Detect and fix vulnerabilities
Admins	Maintain secure systems
Management	Risk visibility

3.2 User Stories

- As a security analyst, I want to scan assets to detect vulnerabilities
 - As a manager, I want reports to understand system risks
-

3.3 Use Cases

- Run vulnerability scan
 - View scan results
 - Generate reports
 - Apply remediation
-

3.4 Functional Requirements

- Perform vulnerability scans
 - Store scan results
 - Generate reports
 - Rank vulnerabilities
-

3.5 Non-Functional Requirements

- High performance
- Secure data handling
- User-friendly interface
- System reliability

4. System Analysis & Design

4.1 Problem Statement & Objectives

Problem:

Organizations often have unidentified vulnerabilities that expose systems to cyber threats.

Objectives:

Develop a system to identify, analyze, and mitigate vulnerabilities efficiently.

4.2 Use Case Diagram & Description

Actors:

- Security Analyst
- System Admin

Interactions:

- Start Scan
 - Analyze Results
 - Generate Reports
-

4.3 Software Architecture

- Scanner Layer
- Processing Layer
- Database Layer
- Visualization Layer

Architecture Style: Layered Architecture

4.4 Database Design & Data Modeling

ER Diagram

Entities:

- Assets

- Vulnerabilities
 - Reports
-

Logical Schema

- Assets (ID, Name, IP)
 - Vulnerabilities (ID, Severity, Description)
 - Reports (ID, Date, Summary)
-

4.5 Data Flow & System Behavior

DFD

1. Input: Scan request
 2. Process: Vulnerability scanning
 3. Output: Reports
-

Sequence Diagram

User → Scanner → Database → Dashboard

Activity Diagram

Start → Scan → Analyze → Report → End

State Diagram

Idle → Scanning → Processing → Completed

Class Diagram

Classes:

- Scanner
 - Vulnerability
 - Report
-

4.6 UI/UX Design & Prototyping

Wireframes

- Dashboard
 - Scan Page
 - Report Page
-

UI/UX Guidelines

- Simple and clean interface
 - Clear visualization (charts/tables)
 - Accessibility considerations
-

4.7 System Deployment & Integration

Technology Stack

- Backend: Python
 - Frontend: Web Dashboard
 - Database: MySQL
-

Deployment Diagram

- User → Web App → Server → Database
-

Component Diagram

- Scanner Component
 - Database Component
 - UI Component
-

4.8 Additional Deliverables

API Documentation (if applicable)

- Endpoint: /scan

- Endpoint: /report
-

Testing & Validation

- Unit testing
 - Integration testing
 - User acceptance testing
-

Deployment Strategy

- Local deployment using virtual machines
 - Optional cloud deployment
-

5. Implementation (Source Code & Execution)

5.1 Source Code

- Clean and structured
 - Proper comments
 - Modular design
-

5.2 Version Control & Collaboration

- GitHub repository
 - Feature branching
 - Clear commit messages
-

5.3 Deployment & Execution

README Includes:

- Installation steps
- Requirements

- Configuration
 - Running instructions
-

Executable / Deployment Link

- Web app or local system access
-

6. Testing & Quality Assurance

Test Plan

- Validate scanning functionality
 - Verify report generation
-

Automated Testing

- Optional scripts
-

Bug Reports

- Document issues and fixes
-

7. Final Presentation & Reports

User Manual

- How to use the system
 - How to run scans
-

Technical Documentation

- Architecture
 - Database
 - APIs
-

Presentation (PPT)

- Problem
- Solution
- Demo
- Results

1. Introduction to the main project

This report presents the results of a vulnerability assessment conducted on a target system within a controlled lab environment. The assessment aimed to identify, analyze, and prioritize security vulnerabilities.

2. Scope

- **Target IP:** 192.168.1.6
 - **Environment:** Virtual Lab (VirtualBox)
 - **Scanner Used:** Nessus Essentials
 - **Scan Type:** Basic Network Scan
-

3. Asset Information

Asset Name	IP Address	OS	Type	Criticality
Metasploitabl e	192.168.1. 6	Ubuntu 8.04 (Linux 2.6)	Interna l	High

```

Last login: Sun May 20 15:50:42 EDT 2012 from 172.16.123.1 on pts/1
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        inet6 ::1/128 scope host
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:26:97:be brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.6/24 brd 192.168.1.255 scope global eth0
        inet6 fe80::a00:27ff:fe26:97be/64 scope link
            valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$ _

```

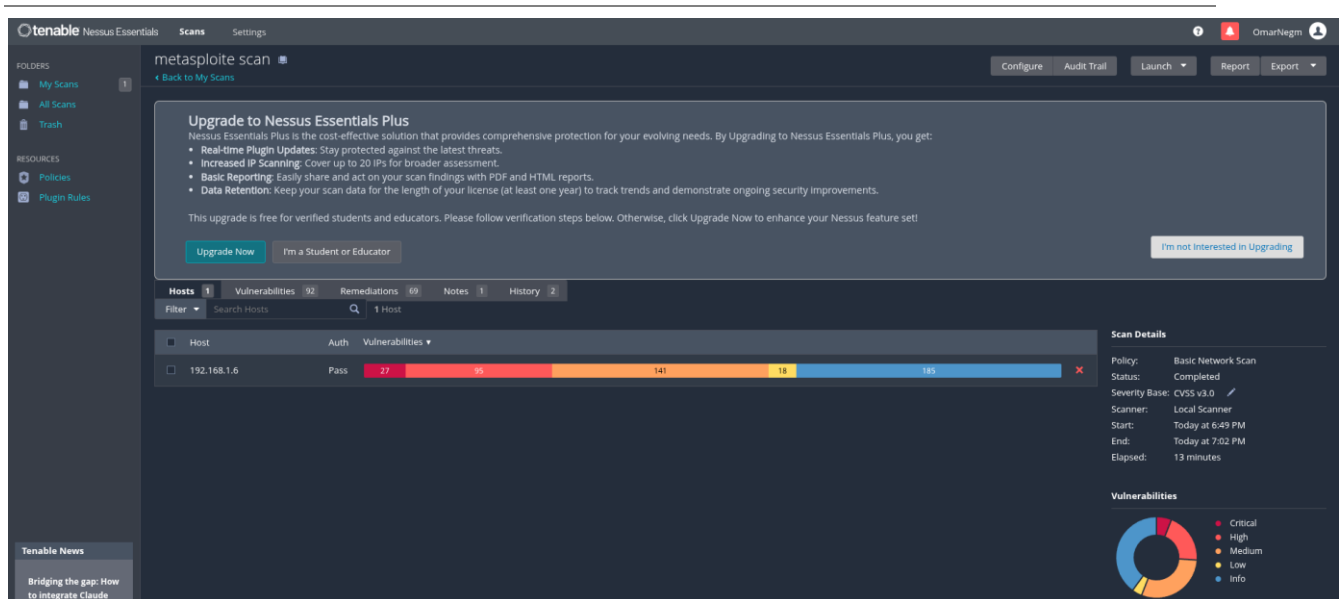
```

negm@negm: ~
(negm@negm)-[~]
$ ping 192.168.1.6
PING 192.168.1.6 (192.168.1.6) 56(84) bytes of data.
64 bytes from 192.168.1.6: icmp_seq=1 ttl=64 time=0.399 ms
64 bytes from 192.168.1.6: icmp_seq=2 ttl=64 time=0.249 ms
64 bytes from 192.168.1.6: icmp_seq=3 ttl=64 time=0.402 ms
64 bytes from 192.168.1.6: icmp_seq=4 ttl=64 time=0.322 ms
64 bytes from 192.168.1.6: icmp_seq=5 ttl=64 time=0.244 ms
64 bytes from 192.168.1.6: icmp_seq=6 ttl=64 time=0.259 ms
64 bytes from 192.168.1.6: icmp_seq=7 ttl=64 time=0.234 ms
64 bytes from 192.168.1.6: icmp_seq=8 ttl=64 time=0.294 ms

```

4. Summary of Findings

Severity	Description
Critical	Multiple remote exploitable vulnerabilities
High	Misconfigurations and outdated services
Medium	Weak encryption and minor issues



5. Detailed Vulnerabilities

5.1 Shellshock vulnerability

- **Severity:** Critical
- **CVSS Score:** 10.0
- **Port:** 22/tcp (ssh)

Description:

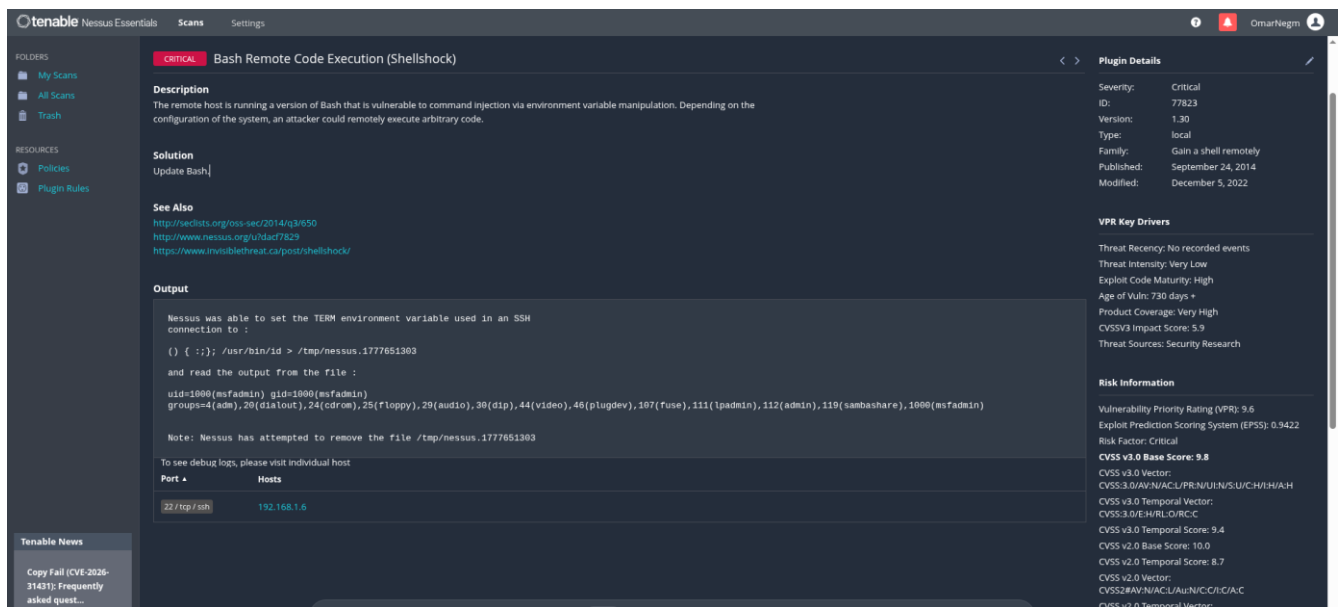
The remote host is running a vulnerable version of Bash that allows attackers to execute arbitrary commands by manipulating environment variables, commonly exploited via SSH, CGI, or other services that pass environment data to Bash.

Impact:

Remote code execution leading to full system compromise, data theft, privilege escalation, and potential lateral movement within the network.

Remediation:

Update Bash immediately to a patched version (4.2.53 or later), and restart any services that rely on Bash environment variable parsing.



5.2 VNC Weak Password:

Severity: Critical

CVSS Score: 10.0

Port: 5900/tcp (vnc)

Description:

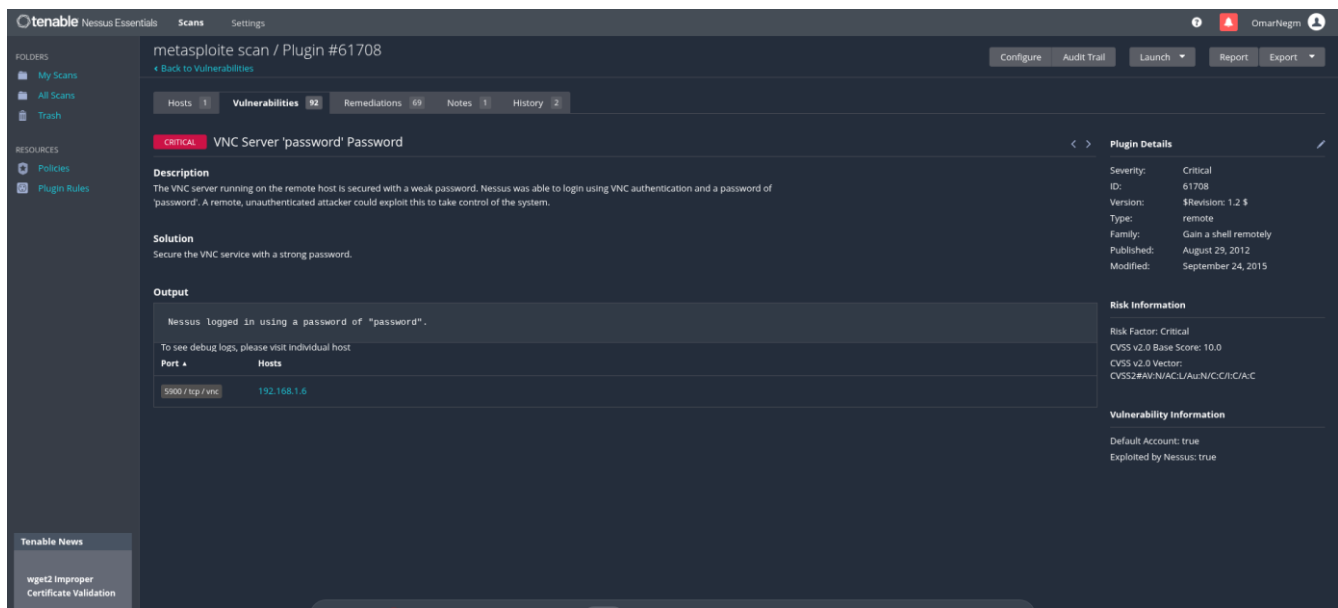
The VNC server running on the remote host is secured with a weak password. Nessus was able to log in using VNC authentication and the password "password". A remote, unauthenticated attacker could exploit this to take control of the system.

Impact:

Full remote takeover of the system, including screen viewing, keyboard/mouse control, file access, and arbitrary command execution.

Remediation:

Secure the VNC service with a strong, complex password (minimum 12 characters, mixed case, numbers, and symbols). Disable VNC if not required, or restrict access using firewall rules and IP whitelisting.



5.3 Apache Tomcat Ghostcat

Severity: High

CVSS Score: 9.8

Port: 8009/tcp (AJP)

Description:

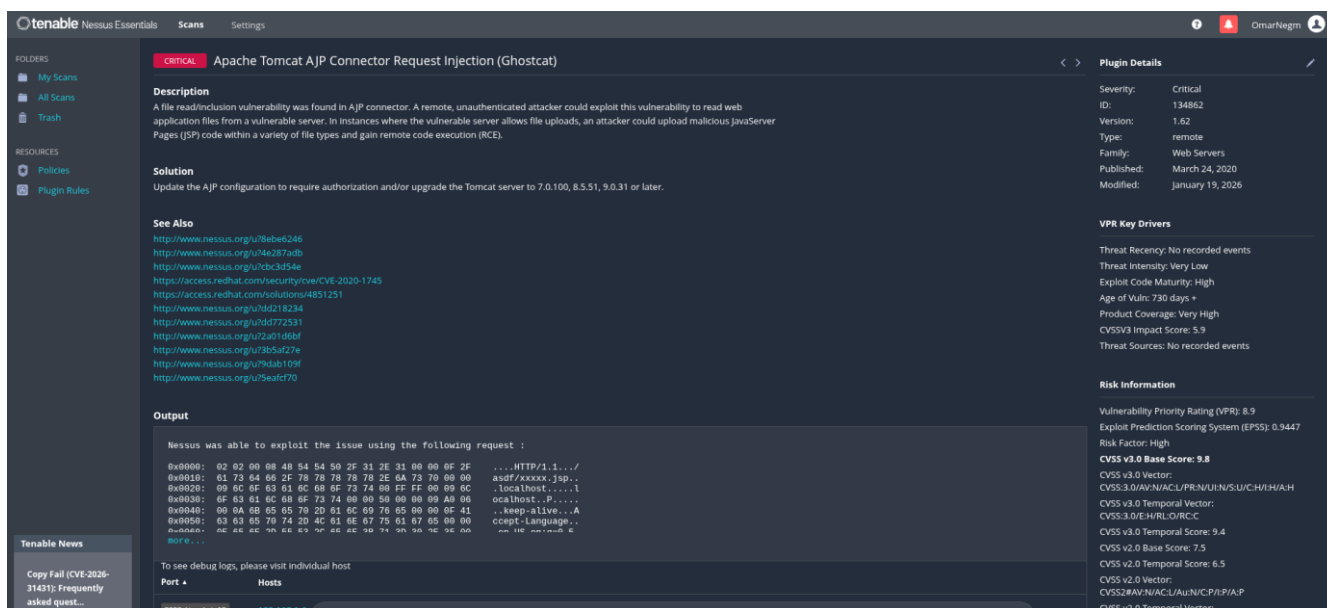
A file read/inclusion vulnerability was found in the AJP connector. A remote, unauthenticated attacker could exploit this vulnerability to read web application files from a vulnerable server. In instances where the vulnerable server allows file uploads, an attacker could upload malicious JSP code within a variety of file types and gain remote code execution (RCE).

Impact:

Unauthenticated attackers can read sensitive files (e.g., configuration files, source code, passwords) from web applications. If file upload functionality exists, attackers can achieve full remote code execution, leading to complete server compromise.

Remediation:

Update the Tomcat server to version 7.0.100, 8.5.51, 9.0.31, or later. Alternatively, update the AJP connector configuration to require authorization, or disable the AJP connector if not in use.



5.4 Weak Debian OpenSSH Keys vulnerability

Severity: Critical

CVSS Score: 10.0

Port: 22/tcp (ssh)

Description:

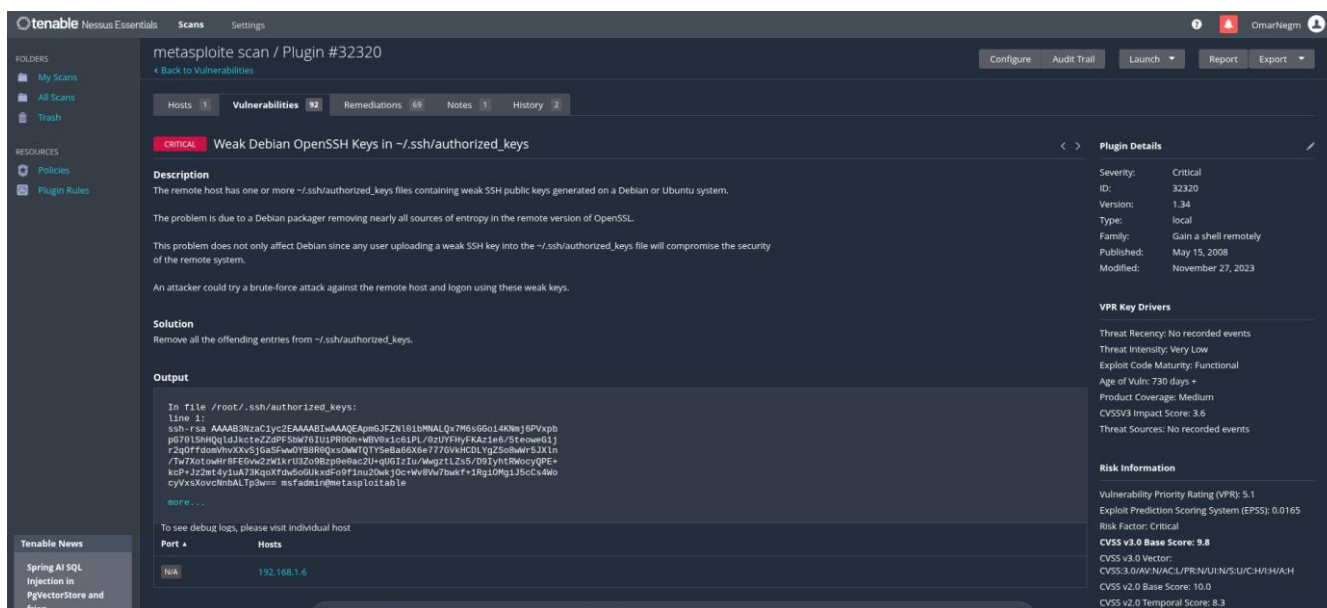
The remote host has one or more `~/.ssh/authorized_keys` files containing weak SSH public keys generated on a Debian or Ubuntu system. The problem is due to a Debian package vulnerability that removed nearly all sources of entropy in the affected version of OpenSSH. This issue affects any system where a user uploads a weak SSH key, regardless of the underlying OS.

Impact:

An attacker could perform a brute-force attack against the remote host and successfully authenticate using these weak keys, leading to unauthorized access, privilege escalation, and full system compromise.

Remediation:

Remove all offending weak key entries from `~/.ssh/authorized_keys` files. Regenerate all SSH keys on any Debian/Ubuntu system that used the vulnerable OpenSSH version, and replace them with securely generated keys.



5.5 SSL Version 2 and 3 Protocol Detection vulnerability

Severity: Critical

CVSS Score: 10.0

Ports: 25/tcp (smtp), 5432/tcp (postgresql)

Description:

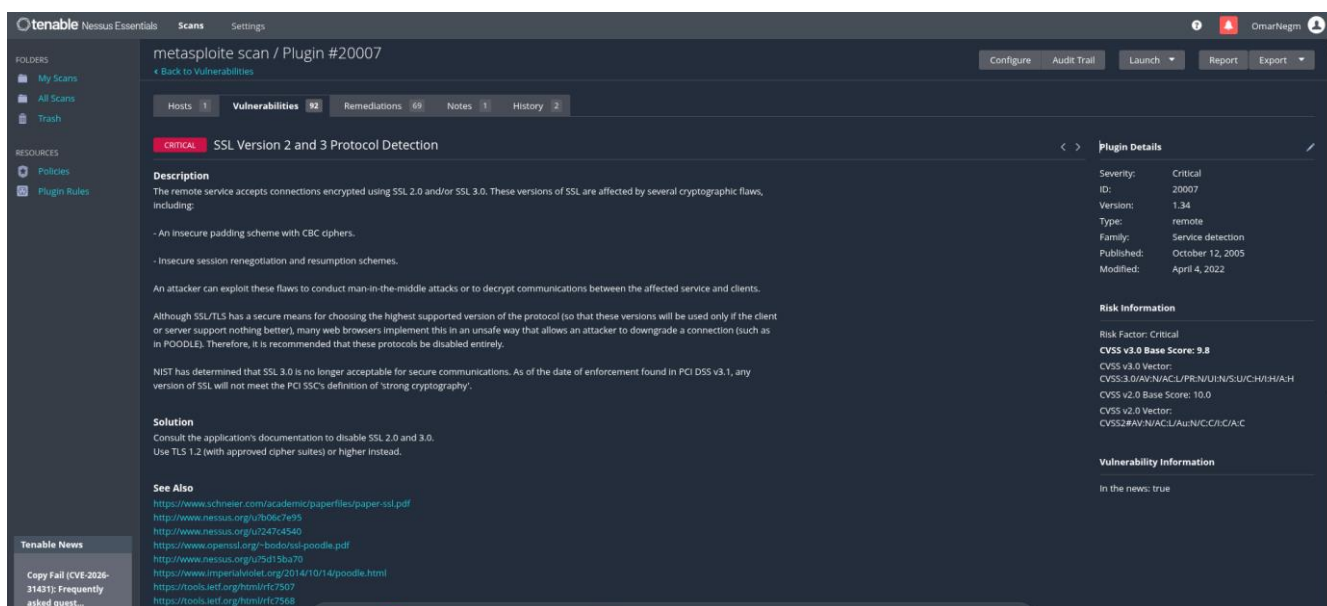
The remote service accepts connections encrypted using SSL 2.0 and/or SSL 3.0. These outdated protocol versions are affected by several cryptographic flaws, including an insecure padding scheme with CBC ciphers, as well as insecure session renegotiation and resumption schemes. Additionally, the remote service supports weak export-grade ciphers such as EXP-RC2-CBC-MD5 and EXP-RC4-MD5 (40-bit key strength).

Impact:

An attacker can exploit these flaws to conduct man-in-the-middle attacks (e.g., POODLE), decrypt communications between the affected service and clients, and potentially recover sensitive data such as credentials or session tokens. The use of export-grade ciphers further weakens encryption to the point where brute-force decryption is feasible.

Remediation:

Disable SSL 2.0 and SSL 3.0 entirely on all affected services. Remove support for weak export-grade and low-strength ciphers. Use TLS 1.2 or higher with strong, modern cipher suites (e.g., AES-GCM, ChaCha20-Poly1305). Consult the application's documentation for specific configuration instructions.



6. Risk Prioritization

CVSS Score	Priority Level	Action Required
9 – 10	Critical	Immediate Fix
7 – 8.9	High	Urgent Fix
4 – 6.9	Medium	Scheduled Fix

7. Remediation Matrix

Vulnerability	Action	Priority
UnrealIRCd	Remove/Update	Critical
VNC	Change Password	Critical
Ghostcat	Patch/Disable AJP	Critical
SSL	Disable old protocols	High
Samba	Update service	High

8. Proof of Validation

- Network scan confirms open ports and active services
 - Vulnerabilities verified using vulnerability scanner
 - Services identified through network enumeration
-

9. Conclusion

The system is highly vulnerable and exposed to multiple critical threats. Immediate remediation is required to prevent potential compromise.

10. Recommendations

- Apply security patches regularly
- Disable unnecessary services
- Use strong authentication mechanisms
- Implement network segmentation
- Conduct periodic vulnerability assessments